



Persistência de Dados em C:

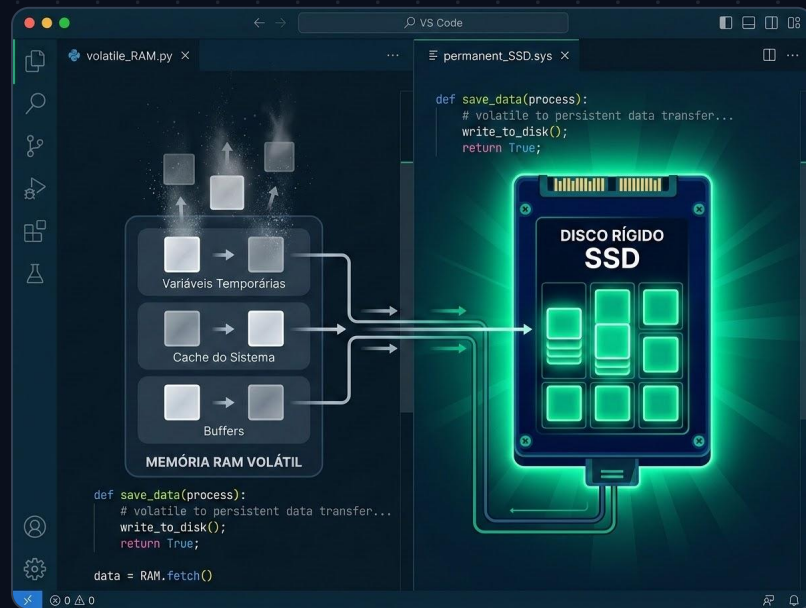
Arquivos e Registros

Manipulação de Arquivos e Registros para Gestão de Notas

```
const char* autor = "Felippe Alex Scheidt";
```

O Desafio da Volatilidade de Dados

- **A Necessidade de Persistência:** Arquivos permitem gravar os dados de um programa de forma permanente.
- **Disponibilidade Contínua:** As informações permanecem disponíveis mesmo após o encerramento do programa, podendo ser restauradas a qualquer momento.
- **Capacidade de Armazenamento:** Permite armazenar grandes quantidades de dados, pois a capacidade de mídias são superiores a capacidade da RAM.
- **Acesso Concorrente:** Vários programas podem acessar um mesmo arquivo permitindo compartilhar uma mesma fonte de dados.



Fundamentos: O que é um Arquivo em C?

Sequência de Bytes: A linguagem C trata os arquivos como uma sequência de bytes que pode ser manipulada por funções específicas.

Arquivos Texto

Armazenam caracteres que podem ser lidos diretamente na tela ou modificados por editores de texto (ex: código fonte C).

Arquivos Binários

São sequências de bits que obedecem a regras específicas do programa que os gerou (ex: executáveis, arquivos compactados).

```
● ● ● declaracao_ponteiro.c
```

```
// Ponteiro FILE: O arquivo é manipulado através de um ponteiro especial definido na biblioteca stdio.h
```

```
FILE *pont_arq;
```

Estrutura: Structs e Vetores de Alunos

- **Agrupamento de Dados:** Um registro (struct) é um tipo de dados definido pelo usuário para representar dados complexos.
- **Entidade Aluno:** Nesse exemplo, vimos que o struct deve armazenar: Nome (até 40 caracteres), Matrícula (inteiro), Nota da Prova 1 (real), Nota da Prova 2 (real), Média (real) e Faltas (inteiro).
- **Situação:** A struct também deve comportar a situação do aluno (aprovado ou reprovado).
- **Vetor de Structs:** Os dados da turma são mantidos em um vetor de structs com capacidade para até 40 alunos.

struct Aluno

char[40]
nome

int
matricula

float / float
nota_p1,
nota_p2

float
media

int
faltas

char[10]
situacao

Abertura e Fechamento de Arquivos

Abertura com `fopen`

- `<ponteiro> = fopen("nome", "tipo");`
- **Modos de Abertura:**
 - escrita binária ("`wb`")
 - leitura binária ("`rb`").
- **Fechamento com `fclose`:** Essencial para garantir que os dados serão salvos.

1 — ABERTURA

```
fopen("arq.bin", "wb");
```



2 — PROCESSAMENTO

```
fread()  
fwrite()
```



3 — FECHAMENTO

```
fclose(pont);
```

⚠ VERIFICAÇÃO DE SEGURANÇA CRÍTICA

→ A função `fopen()` retorna **NULL** se não conseguir abrir o arquivo (*falta de permissão, inexistente,...*).

→ Sempre testar o retorno: `if (pont_arq == NULL) { ... }` antes de executar qualquer leitura ou escrita.

Salvando Registros com `fwrite`

A função `fwrite()` grava blocos de dados de tamanho fixo diretamente em um arquivo binário.

```
size_t fwrite(  
    const void *ap_dados,  
    size_t tam_registro,  
    size_t num_registros,  
    FILE *ap_arquivo  
);
```

Retorno: número
de itens gravados.

Parâmetros:

1. endereço dos dados
2. tamanho do registro em bytes (`sizeof`)
3. quantidade de registros a gravar
4. ponteiro do arquivo.

● ● ● exemplo_fwrite.c

```
// Definição da struct Aluno  
typedef struct {  
    int rga;  
    char nome[50];  
} Aluno;  
  
// 1. Abre o arquivo no modo de escrita binária  
FILE *arq = fopen("alunos.bin", "wb");  
  
Aluno a1 = {2026, "Maria"};  
  
// 2. Salva o registro no arquivo  
size_t qtd = fwrite(  
    &a1,  
    sizeof(Aluno),  
    1,  
    arq  
);  
  
// 3. Fecha o fluxo do arquivo  
fclose(arq);
```



Leitura Binária com fread()

fread: Lê blocos de dados de tamanho fixo de um arquivo binário e os armazena na memória RAM.

```
size_t fread(void *ap_dados,  
             size_t tam_reg,  
             size_t num_reg,  
             FILE *ap_arq  
);
```

Funcionamento: Lê o número de registros especificado (num_reg), cada um com o tamanho (tam_reg) definido em bytes, a partir do arquivo.

Retorno: o número de itens lidos com sucesso, permitindo o controle de quantos alunos foram efetivamente carregados para o vetor.

● ● ● exemplo_fread.c

```
// Carregando vetor de structs do disco  
struct Aluno vetor[40];  
FILE *arq = fopen("alunos.bin", "rb");  
if (arq != NULL) {  
    size_t lidos = fread(  
        vetor,  
        sizeof(struct Aluno),  
        40,  
        arq  
    );  
  
    // Controla registros lidos  
    printf("Carregados: %zu\n", lidos);  
    fclose(arq);  
}
```

Exercício: Registro de notas

// REQUISITOS DO SISTEMA

Diretrizes do Projeto

Objetivo:

Implementar um programa para tratar e armazenar as notas de até 40 alunos.

Menu Principal:

O programa deve rodar em um loop oferecendo 5 opções ao usuário.

● ● ● sistema_gestao.app

```
$ ./sistema_gestao.app
```

```
--- MENU PRINCIPAL ---
```

1. **Opção 1:** Inserir alunos (ler dados, calcular média e situação).
2. **Opção 2:** Exibir alunos (imprimir dados na tela).
3. **Opção 3:** Salvar dados (gravar vetor em arquivo).
4. **Opção 4:** Carregar dados (ler arquivo para a memória).
5. **Opção 5:** Sair do programa.

```
Digite sua opção [1-5]: _
```


Opção [1] -> Lógica para Inserção e Cálculo

- **Entrada de Dados:** A função deve ler via teclado o nome, matrícula, notas (Prova 1 e Prova 2) e o número de faltas do aluno.
- **Cálculo Automático:** O programa deve calcular a média real com base nas duas provas informadas.
- **Atribuir um valor para a situação:**
 - **Aprovado:** Se a média for ≥ 6 e as faltas forem ≤ 20 .
 - **Reprovado:** Se a média for < 6 ou as faltas forem > 20 .
- **Armazenamento:** Os dados processados são mantidos no vetor da turma na memória.

calcula.c

```
// Cálculo da média aritmética
aluno.media = (aluno.p1 + aluno.p2) / 2.0;

// Definição da situação do aluno
if (aluno.media >= 6.0 && aluno.faltas <= 20)
{
    strcpy(aluno.situacao, "Aprovado");
}
else {
    strcpy(aluno.situacao, "Reprovado");
}

// Armazena no vetor da turma
turma[index] = aluno;
```

Opção [2] -> Imprimir dados dos Alunos

Listagem de Dados

Esta função é responsável por imprimir na tela os dados de todos os **alunos** armazenados no vetor **turma**.

Varredura do Vetor

Utiliza-se um laço de repetição (**for**) para navegar sequencialmente por todos os registros

Acesso aos Membros

Para cada índice, o programa acessa e formata campos da struct
(ex: **turma[i].nome** e notas).

 `exibir_alunos.c`

```
// Percorre a turma e exibe formatado
void listarAlunos(Aluno turma[], int total) {
    for (int i = 0; i < total; i++) {
        printf("Matrícula: %d\n", turma[i].matr);
        printf("Nome: %s\n", turma[i].nome);
        printf("Média: %.1f\n", turma[i].media);
        printf("Situação: %s\n", turma[i].sit);
    }
}
```

Opções [3] e [4] - Salvar e Carregar Dados

Opção 3: Salvar

1. **fopen**: Abre o arquivo em modo de escrita binária ("wb").

2. **fwrite**: Grava todo o vetor de alunos de uma só vez no arquivo.

3. **fclose**: Fecha o arquivo de forma segura e encerra a operação.

Opção 4: Carregar

1. **fopen**: Abre o arquivo em modo de leitura binária ("rb").

2. **fread**: Puxa os dados do arquivo de volta para a estrutura na memória.

3. **fclose**: Fecha o arquivo e encerra as operações de leitura.

persistencia_alunos.c

```
// Opção 3: Escrita Binária (Salvar)
FILE *arq = fopen("alunos.bin", "wb");
if (arq != NULL) {
    fwrite(alunos,
           sizeof(Aluno),
           total,
           arq);
    fclose(arq);
}

// Opção 4: Leitura Binária (Carregar)
FILE *arq = fopen("alunos.bin", "rb");
if (arq != NULL) {
    fread(alunos, sizeof(Aluno), total, arq);
    fclose(arq);
}
```

Conclusão: Boas Práticas

"Abriu, Verificou, Usou, Fechou!"

Tratamento de Erros → Sempre verifique se o ponteiro retornado por **fopen** é igual a **NULL** para evitar falhas silenciosas na leitura ou gravação.

Fechamento Obrigatório → O uso do **fclose** garante que os buffers do sistema operacional sejam descarregados e que o arquivo seja efetivamente salvo no disco.

Atividade

Defina a `struct` do aluno, crie o menu principal em loop e implemente as funções de manipulação de vetor (1 até 5), com leitura e escrita de um arquivo que mantém informações dos alunos.

// REQUISITOS DO SISTEMA

Diretrizes do Projeto

Objetivo:

Implementar um programa para tratar e armazenar as notas de até 40 alunos.

Menu Principal:

O programa deve rodar em um loop oferecendo 5 opções ao usuário.

● ● ● sistema_gestao.app

```
$ ./sistema_gestao.app
```

```
--- MENU PRINCIPAL ---
```

1. **Opção 1:** Inserir alunos (ler dados, calcular média e situação).
2. **Opção 2:** Exibir alunos (imprimir dados na tela).
3. **Opção 3:** Salvar dados (gravar vetor em arquivo).
4. **Opção 4:** Carregar dados (ler arquivo para a memória).
5. **Opção 5:** Sair do programa.

```
Digite sua opção [1-5]: _
```